

Bridging the Last Mile of Circuit Design: POSTEDA-BENCH, a Hierarchical Benchmark for PPA Convergence and DRC Fixing

Pengju Liu¹, Nuo Xu¹, Jinwei Tang¹, Yu Cao¹, Caiwen Ding¹

¹University of Minnesota
liu03486@umn.edu

Abstract

LLM-based agents are increasingly applied to the “last mile” of Electronic Design Automation (EDA): repairing residual sign-off Design Rule Check (DRC) violations and converging Power–Performance–Area (PPA) targets after tool runs. Existing EDA-LLM benchmarks, however, omit DRC fixing entirely and rely on flat hierarchies tied to a single toolchain. We introduce POSTEDA-BENCH, a hierarchical benchmark with 145 tasks across **DRC-Essential**, **DRC-Reasoning**, **PPA-Mono**, and **PPA-Multi**, supported by EDA toolchains with machine-checkable evaluation. Across eight commercial and open-source LLMs under multiple agent scaffolds, we find that agents handle synthetic DRC-Essential and single-objective PPA-Mono reasonably well but degrade sharply on the more practical DRC-Reasoning (best SR 36.66%) and PPA-Multi (best SR 20.00%); vision augmentation consistently enhances DRC-Bench; and trade-off reasoning—rather than knob knowledge—is the dominant PPA-Multi bottleneck. Our benchmark is open-sourced at <https://github.com/pengjas/posteda-bench>.

1 Introduction

Large language model (LLM)–based agentic systems have demonstrated strong performance in code generation[7, 22, 1], robotics and embodied AI[21, 3], and mathematical reasoning[15, 5]. These advances have motivated recent work on LLM-driven agents for automating power–performance–area (PPA) convergence in electronic design automation (EDA) flows [18, 19, 17, 12, 8, 4, 10, 6, 12], where engineers often spend significant time iterating after tool runs to meet PPA targets.

Despite this progress, existing EDA-agent benchmarks still **fall short of what is needed to evaluate practical post-EDA closure**. As summarized in Table 1, most benchmarks either focus on **NLP-to-script generation rather than target-driven PPA optimization**, contain only a small number of optimization tasks, use flat or coarse task hierarchies, or are tied to a single toolchain, limiting both diagnostic resolution and generality. More importantly, **none of them evaluates design rule check (DRC) fixing**, although residual sign-off DRC violations remain a time-critical “last-mile” workload even after modern routing. These gaps make a unified and realistic benchmark for post-EDA PPA optimization and DRC repair both **urgent and necessary**.

To fill this gap, we introduce **POSTEDA-BENCH**, a hierarchical benchmark for evaluating LLM-based agents on post-EDA closure: **POSTEDA-BENCH**: 145 tasks across two dimensions, **DRC-Bench** and **PPA-Bench**, evaluating whether an agent can (i) repair sign-off DRC violations and (ii) drive designs to specified PPA targets, supported by both open-source (OPENROAD) and commercial (DC+INNOVUS) toolchains with machine-checkable evaluation. **DRC-Bench**: splits into DRC-Essential, three levels probing rule knowledge, contextual robustness, and cascaded multi-violation fixing on synthetic cases, and DRC-Reasoning, three levels of practical residual post-flow violations

Table 1: Comparison with related benchmarks.

Benchmark	#Tasks	Open -sourced	Objective	Hierarchy	EDA Toolchains	DRC Fix
ChatEDA-bench [18]	50	✓	NLP→scripts	Simple–Complex	OpenROAD	✗
GPT4AIGChip [4]	6	✗	Codegen&PPA Opt.	Flat	Vivado HLS tools	✗
Chipnemo [10]	959	✗	NLP→scripts	Easy–Medium–Hard	n/a	✗
IICPilot [8]	4	✗	PPA Opt.	Flat	OpenROAD	✗
iEDA-bench [19]	50	✗	NLP→scripts	Simple–Complex	OpenROAD	✗
MCP4EDA [17]	10	✓	NLP→scripts	Flat	OpenROAD	✗
ORFS-agent [6]	6	✓	PPA Opt.	Flat	OpenROAD	✗
AutoEDA [12]	100	✗	NLP→scripts	Different–stage in EDA	DC&Innovus	✗
POSTEDA-BENCH	145	✓	PPA Opt.	Multi-dimension& Multi-level	OpenROAD& DC&Innovus	✓

†: NLP→scripts: generates EDA scripts from natural language; it runs tools but doesn’t optimize PPA.

§: PPA Opt. denotes PPA Optimization. *: OpenROAD is open-source, whereas DC and Innovus are commercial tools.

‡: DC and Innovus are only used to generate part of DRC-Bench tasks in POSTEDA-BENCH.

that demand geometric reasoning, stratified by minimum required edit steps. **PPA-Bench**: splits into PPA-Mono (Performance, Power, Area), with single- and multi-knob perturbation levels plus a Performance re-architecture level, and PPA-Multi, with 2D and 3D Pareto-targeted tasks that test trade-off reasoning under constraint floors.

Using POSTEDA-BENCH, we evaluate commercial and open-source models, frontier and small backbones, and text-only versus vision–language variants under several agent scaffolds (ReAct, Proposer–Critic, ORFS-Agent, Reflexion and ToT). **Our key findings are as follows**:

- ▷ **Synthetic vs. practical asymmetry.** Agents perform reasonably well on synthetic DRC-Essential (best SR 85.50%) and single-objective PPA-Mono (best SR 64.56%), but degrade sharply on the more practical DRC-Reasoning (best SR 36.66%) and PPA-Multi (best SR 20.00%), where post-flow geometric reasoning and multi-objective trade-offs dominate.
- ▷ **Vision augmentation enhances DRC-Bench.** Adding a layout-image channel is never harmful and yields consistent SR/VRR gains on both DRC-Essential and DRC-Reasoning, with the largest lifts when text-only baselines are weak.
- ▷ **Trade-off reasoning is the PPA-Multi bottleneck.** Several backbones produce *negative* NIS on PPA-Multi by regressing constraint metrics while chasing the violated objective, indicating they greedily optimize one PPA dimension rather than balancing competing targets.

2 Related Work

DRC fixing. Design rule checking verifies layout manufacturability, yet residual sign-off errors and cascaded fixes often remain after automated repair. This motivates evaluating agents on rule understanding, layout inspection, geometric editing, and iterative DRC checking; additional background is in Section A. **PPA optimization.** Power, performance, and area are coupled objectives, and automated flows often miss tight targets despite many configuration, constraint, and design-level knobs. PPA-Bench targets this last-mile loop by testing whether agents can use tool feedback for single-objective improvement and multi-objective trade-offs; extended context is in Section A.

3 POSTEDA-BENCH

Preliminary Source Design Collection. As shown on the left of Figure 2, POSTEDA-BENCH starts from three public RTL sources: RTLLM v2 [11] (50 designs), VERILOGEVAL-Human [9] (156 problems), and OPENCORES [14] (1,283 listed IP projects), covering arithmetic circuits, controllers, and community IP blocks. We map retained designs to ASAP7 [2] and keep only candidates satisfying four auditable criteria: (i) **parse and synthesis validity**, i.e., a well-defined top module, valid syntax, and successful gate-level synthesis; (ii) **non-trivial scale**, with more than 300 instantiated standard cells; (iii) **cross-source deduplication** by top-module names, port signatures, and post-synthesis structural similarity, merging designs with more than 90% gate-level overlap; and (iv) **release and audit eligibility**, requiring documented provenance, license, selected top module, and derived artifacts. Aggregate statistics are reported in Table 2; Section B details deduplication, contamination control, and the release package.

Table 2: **Statistics of POSTEDA-BENCH**

Statistic	Number
Tasks	145
Std. cells per source design	min 301; avg 16,505; max 95,061
Covered layers in DRC-Bench	N(P)Well – M9
DRC-Essential rules (before pruning)	168
Human fix steps (DRC-Reasoning)	min 1; max 8
DRC runtime [†]	min 10 s; max 3 h
Needed Tunable parameters [§]	min 1; max 7
Mono-Perf tasks requiring re-architecture [*]	4

[†]: DRC runtime is the wall-clock time for one signoff DRC check; it reflects design-scale diversity and is not agent or human repair time.

[§]: Needed tunable parameters are minimum critical knobs; agents may edit all OpenROAD parameters.

^{*}: Mono-Perf re-architecture tasks require RTL/micro-architectural changes beyond parameter tuning.

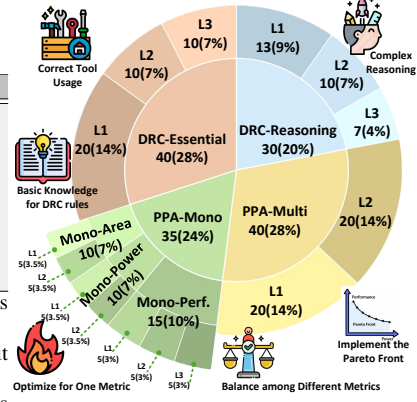


Figure 1: **Overview of POSTEDA-BENCH composition.**

Because public RTL may appear in model pretraining corpora, POSTEDA-BENCH **controls contamination** by evaluating downstream tool interaction, benchmark-specific targets, and generated EDA artifacts rather than source-code recall. For **reproducibility and release**, both DRC-Bench and PPA-Bench ship with inputs, prompts, metadata, pinned tool setup, and evaluation drivers; final labels are machine-checkable through deterministic EDA tools and report parsers. Detailed release contents, tool versions, regeneration requirements, and limitations are documented in Sections B to D and H.

3.1 DRC-Bench

DRC-Bench evaluates last-mile sign-off DRC repair, a workload absent from existing EDA-LLM benchmarks (Table 1), through two branches: **DRC-Essential**, which isolates rule knowledge, layout-tool use, context robustness, and cascaded fixing on synthetic cases, and **DRC-Reasoning**, which targets residual post-flow violations requiring geometric reasoning.

3.1.1 Task Specifications

Input. Each task provides two artifacts: the target .gds layout and a natural-language prompt (Figure 3a). The prompt includes the initial sign-off DRC violations, with each entry specifying the rule identifier (e.g., V0.M1.AUX.3), a short rule description, the geometry type (edge-pair, polygon, or edge), and layout coordinates. **Metrics.** A run is **successful** when the final layout is DRC-clean under sign-off DRC, and we report the **Success Rate (SR)**, computed as the mean of the success indicator over 5 independent runs per task. To capture partial progress, we additionally report the **Violation Reduction Rate (VRR)**:

$$\text{VRR} = \frac{\max(0, E_{\text{initial}} - E_{\text{final}})}{E_{\text{initial}}} \times 100\%, \quad (1)$$

where E_{initial} and E_{final} denote the total violation count before and after the agent’s edits. The choice of sign-off tool and DRC deck is documented in Section C. **Tools.** Agents can query layout context textually or visually, edit GDS geometry and placement, and rerun sign-off DRC through a compact layout-viewer-style tool set; full schemas are in Section C.3.

3.1.2 DRC-Essential

DRC-Essential has three levels (light-apricot block in Figure 2), each isolating a primitive competency. **L1 (Atomic rule understanding).** L1 uses atomic single-rule violations with only the required shapes. Cases are hand-drawn from the ASAP7 design-rule manual or generated by per-task scripts (create_errors.py; Section C.4). We group *topologically equivalent* rules—those differing only in layer assignment—into one representative per class, balanced to one case per layer. **L2 (Contextual comprehensiveness).** L2 embeds one L1-style violation in realistic layout context, forcing agents to filter relevant geometry from clutter. Construction: (i) run a full EDA flow—OPENROAD-FLOW-SCRIPTS or SYNOPSIS DC +CADENCE INNOVUS—on a source-pool design; (ii) have an engineer repair residuals to a DRC-clean reference; (iii) inject one L1-style violation whose fix touches only

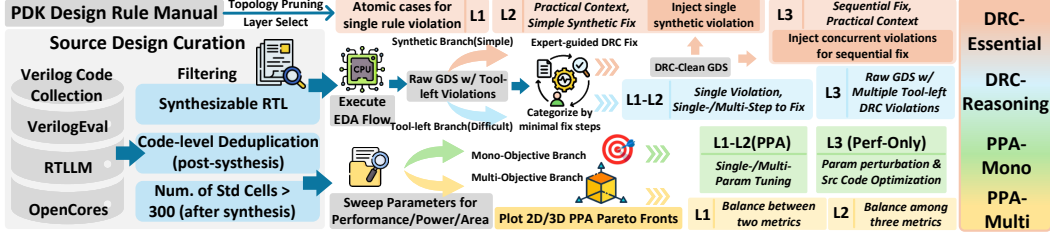


Figure 2: Overview of POSTEDA-BENCH construction process.

the offending shapes. L2 thus isolates context filtering without adding geometric reasoning. **L3 (Sequential fixing)**. L3 tests whether agents can resolve interacting violation chains. For each DRC-clean design we inject 5–15 L1-style violations across rule families, mainly width/via (e.g., M2.W.1, M3.W.1, etc.). Sites are co-located so edits interact; each violation is single-step in isolation, but agents must order edits and re-query DRC since fixes can trigger or eliminate nearby ones.

3.1.3 DRC-Reasoning

DRC-Reasoning targets practical residual violations that remain after full EDA flows and built-in repair (blue block in Figure 2; Section C.5 quantifies the residual gap for Innovus). These cases are harder than DRC-Essential because they require non-trivial geometric reasoning. We partition them into three levels by the *minimum number of agent editing steps* required (one step = one `add_shape/change_shape/move_cell` call; inspection not counted). Labels come from an experienced physical-design engineer following the protocol in Section C.6, with a delayed self-consistency pass. **L1–L2 (Single- vs. multi-step fixes)**. We sweep OPENROAD and DC+INNOVUS for post-flow residual violations across open-source and industrial settings. A violation is **L1** if its repair takes one editing step, **L2** if multiple (no upper cap). When unrelated violations co-exist, we isolate the target by cropping or temporarily fixing surroundings, so L1/L2 cleanly probe single-violation reasoning. **L3 (Multi-violation debugging)**. L3 reproduces the practical multi-violation scenario: each task is a raw post-flow GDS with several real violations that the agent must iteratively debug and resolve. L3 designs may overlap with those seeding L1/L2 (different post-flow configs of the same source), but the per-task violation set is disjoint by construction.

3.2 PPA-Bench

Beyond DRC, post-tool closure often fails because designs miss PPA (power, performance, and area) constraints. PPA-Bench contains two subsets: **PPA-Mono** optimizes one metric (performance, power, or area), while **PPA-Multi** evaluates trade-offs among multiple metrics.

3.2.1 Task Specifications

Inputs. Each task is a self-contained OpenROAD project with Verilog sources (`src/`), configs (`config.mk`, `constraints.sdc`), and previous-run artifacts (`logs/`, `reports/`, `objects/`, `results/`). The prompt summarizes current PPA, target thresholds, constraint floors, and the `FlowVariables.md` knob list (Figure 3b); ground-truth metadata is access-controlled in Section D.2. **Metrics.** A run is **successful** only when every target metric is met and the modified RTL preserves **functional equivalence** with the original design under our testbench-based check (Section D.5); **Success Rate (SR)** is the mean success fraction over 5 temperature-0 runs per task. We also report

```
the input gds file is ./top_cell.gds
Top Cell: aes_cipher_top
=====
--- V0.M1.AUX.3 | cell: aes_cipher_top
[0] {'category': 'V0.M1.AUX.3',
'description': 'V0 must exactly be the
same width as M1 along the Direction ...
```

(a) Sample prompt in DRC-Bench.

```
The design has completed the OpenROAD
flow, but effective period is not met.
Current effective period: 212.2 ps
Target effective period: <= 210.0 ps
Analyze any available files and optimize
design configuration or timing
constraints in `conf/.mk`
```

(b) Sample prompt in PPA-Bench.

Figure 3: Example prompts used to drive the agent in DRC-Bench and PPA-Bench.

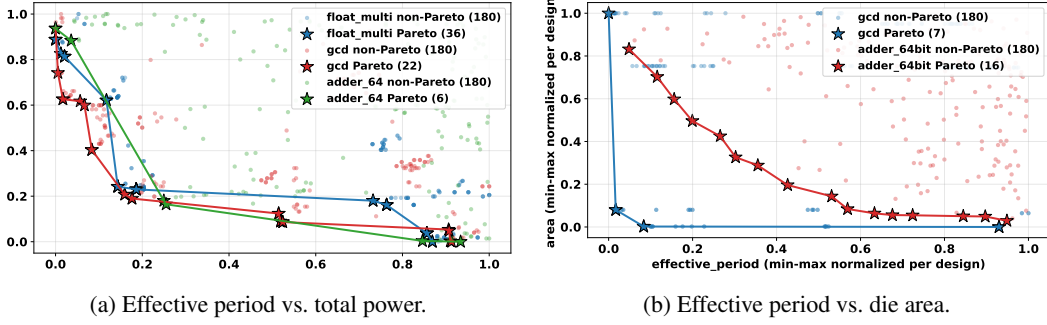


Figure 4: **Two-objective Pareto fronts for representative PPA-Multi designs.** Gray dots denote non-Pareto solutions; blue stars indicate Pareto-optimal points. The combined panels summarize representative period–power and period–area trade-off fronts.

the **Normalized Improvement Score (NIS)** for partial progress. For each constrained metric $i \in \mathcal{M}$, let $\mathcal{A} = \{i : M_i^{\text{init}} > M_i^{\text{tgt}}\}$ be the violated metrics to improve and $\mathcal{B} = \{i : M_i^{\text{init}} \leq M_i^{\text{tgt}}\}$ be constraints that must not regress; the per-metric score is

$$s_i = \begin{cases} \min(1, r_i), & i \in \mathcal{A}, \\ 1, & i \in \mathcal{B} \text{ and } M_i^{\text{fin}} \leq M_i^{\text{tgt}}, \\ 1 - r_i, & i \in \mathcal{B} \text{ and } M_i^{\text{fin}} > M_i^{\text{tgt}}, \end{cases} \quad \text{where } r_i = \frac{M_i^{\text{fin}} - M_i^{\text{init}}}{M_i^{\text{tgt}} - M_i^{\text{init}}}. \quad (2)$$

The aggregate score is then

$$\text{NIS} = \begin{cases} 0, & \mathcal{A} \neq \emptyset \text{ and } M_i^{\text{fin}} \geq M_i^{\text{init}} \forall i \in \mathcal{A}, \\ \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} s_i, & \text{otherwise.} \end{cases} \quad (3)$$

All metrics are lower-is-better: $r_i = 1$ means the target is reached, $r_i \in (0, 1)$ means partial progress, and $r_i < 0$ means regression. For single-objective tasks ($|\mathcal{M}| = 1$) we clip s_1 to $[0, 1]$; for multi-objective tasks, negative scores penalize regression, and Eq. 3’s zero-floor rule prevents trade-off exploits that improve only constraint metrics. Per-task scores are averaged within each class. **Run protocol.** Each run is bounded by a tool-call iteration cap, treated as an evaluation hyperparameter and varied in the iteration ablation. **Tools.** Agents inspect files, edit configuration/constraints/RTL, run OpenROAD in an isolated container, and parse PPA reports through four sandboxed tools; full schemas and locked variables are in Section D.4.

3.2.2 PPA-Mono

PPA-Mono evaluates single-objective optimization across **Mono-Performance**, **Mono-Power**, and **Mono-Area** (green block in Figure 2), with no constraints on the other two metrics so each task isolates one objective. **Construction.** For each source design, we grid-sweep a curated subset of OpenROAD-flow-scripts variables (e.g., ABC_AREA, ASAP7_USE_VT, CORE_UTILIZATION, PLACE_DENSITY, SYNTH_HIERARCHICAL; full list in Section D.3) and identify the configuration with the best target-metric value. This configuration defines the reference fix and task target; task instances are then created by controlled perturbations that move the design *away* from the reference. **L1–L2 (single- vs. multi-knob perturbation).** **L1** perturbs one uniformly sampled knob to another reasonable value that worsens the target metric. **L2** jointly perturbs several knobs, typically 2–3, under the same degradation constraint. The agent is *not* told which knobs were changed; it observes only the degraded post-flow state and must localize responsible parameters through tool calls. **L3 (Performance-only re-architecture).** Some performance gaps cannot be closed by knob retuning and require source-level restructuring, such as pipelining, retiming, or SDC changes. We restrict L3 to Performance because Power and Area gaps in our pool are generally configuration-closable.

3.2.3 PPA-Multi

PPA-Multi evaluates multi-objective tuning under explicit metric trade-offs (light-yellow block in Figure 2). For each design we conduct the same grid sweep used in PPA-Mono, and for each task we

Table 3: **Overall performance on POSTEDA-BENCH.** For both SR and VRR/NIS, all values are in % and *larger is better*. Within each column, the best score is shown in **bold** and the second-best is underlined; tied entries share the same rank. “–” indicates that the framework does not apply to that dimension (ORFS-Agent targets PPA only).

	DRC-Essential		DRC-Reasoning		PPA-Mono		PPA-Multi	
	SR(%)	VRR(%)	SR(%)	VRR(%)	SR(%)	NIS(%)	SR(%)	NIS(%)
<i>React</i>								
GPT-5	84.50	91.94	31.33	33.80	39.42	51.59	11.50	23.69
GPT-5-mini	65.00	75.21	20.66	21.88	42.28	47.74	10.00	17.36
Gemini-3-Flash-preview	85.50	87.78	36.66	36.75	48.56	54.16	18.50	51.62
DeepSeek-V3.2	71.00	79.78	16.66	18.64	1.14	7.84	0.00	1.33
Qwen3.5-122B-A10B	51.50	58.97	13.33	14.09	39.42	47.84	4.00	35.46
Gemma-4-31B-it	82.00	92.02	27.99	32.05	64.56	69.34	11.00	16.34
Qwen3.5-27B	68.50	77.66	27.32	30.07	30.85	42.58	0.0	-39.34
Qwen3.5-9B	56.00	64.85	5.99	5.99	25.71	36.56	1.00	10.88
<i>Proposer-Critic</i>								
GPT-5	60.50	71.57	15.99	17.00	42.85	50.36	0.50	11.79
Qwen3.5-122B-A10B	66.50	75.27	17.33	18.89	39.99	54.16	2.50	25.03
Gemma-4-31B-it	78.00	87.00	31.33	33.51	42.28	49.95	2.00	-1.33
Qwen3.5-9B	54.50	62.58	8.66	9.18	3.99	13.61	0.00	-16.54
<i>Domain-specific Agent (ORFS-agent[‡])</i>								
GPT-5	–	–	–	–	25.71	33.75	5.00	50.28
Qwen3.5-122B-A10B	–	–	–	–	40.00	48.91	20.00	67.80
Gemma-4-31B-it	–	–	–	–	47.14	47.31	1.25	-15.90

§: We exclude other open-sourced ChatEDA[18] and MCP4EDA[17] that generates scripts from prompts.

‡: We automate ORFS-Agent search-space inference by removing manual parameter/range definitions.

set targets at points selected from the Pareto frontier so that they are jointly achievable but inaccessible by greedy single-metric optimization. **L1 (two-objective trade-off)**. Each task targets a point on a 2D Pareto front. We cover all three metric pairs—performance–power, performance–area, and power–area—so the corpus jointly stresses every pairwise trade-off (Figure 4 illustrates representative combined 2D fronts). **L2 (three-objective trade-off)**. Each task targets a point on the full 3D Pareto front spanning performance, power, and area; targets are chosen from the frontier to ensure achievability while precluding shortcut solutions. In both levels the initial configuration sits in the off-frontier interior, so the agent must *reach the front from the interior* while satisfying floor constraints on all non-target metrics: the score in Eq. 3 explicitly penalizes regressions on those constraints. PPA-Multi therefore tests whether agents can reason about coupled parameters and balance non-trivial trade-offs, rather than greedily optimize a single dimension.

4 Experiments

4.1 Experimental Setup

Models. We evaluate three commercial models (GPT-5, GPT-5-mini, Gemini-3-Flash-preview) and five open-source models spanning MoE (DeepSeek-V3.2, Qwen3.5-122B-A10B), mid-size dense (Qwen3.5-27B, Gemma-4-31B-it), and small dense (Qwen3.5-9B). **Agent frameworks.** We compare **ReAct** [21], **Proposer-Critic** [13], and an adapted **ORFS-Agent** [6] (Bayesian optimization, PPA only; see Section E). **Run protocol.** Each triple runs 5 times at temperature 0; we report mean SR/VRR/NIS. ReAct and Proposer-Critic use adaptive caps of 8 DRC / 16 PPA agent actions; ORFS-Agent uses 18 OpenROAD runs per PPA task (Sections E.2 and E.3).

4.2 Main Results

Table 3 shows no dominant model. Gemini-3-Flash-preview leads DRC-Essential SR (85.50%) and DRC-Reasoning SR/VRR (36.66%/36.75%) and ranks second on both PPA dimensions; Gemma-4-31B-it leads DRC-Essential VRR (92.02%) and PPA-Mono SR/NIS (64.56%/69.34%); ORFS-wrapped Qwen3.5-122B leads PPA-Multi (20.00%/67.80%). GPT-5 wins no column but is strong on DRC, while DeepSeek-V3.2 drops from 71.00% DRC-Essential SR to near-zero PPA. **Synthetic-to-practical drop is the dominant gap:** agents perform reasonably on the synthetic dimensions (best

Table 4: **DRC-Bench per-level performance.** SR/VRR for representative ReAct backbones on L1–L3; parenthesized Δ values report the difference from the corresponding overall DRC score. The best score in each column is bolded, with ties sharing bold.

	L1		L2		L3	
	SR(%)	VRR(%)	SR(%)	VRR(%)	SR(%)	VRR(%)
<i>DRC-Essential</i>						
GPT-5	94.00 (+9.50)	94.00 (+2.06)	96.00 (+11.50)	97.00 (+5.06)	54.00 (−30.50)	82.78 (−9.16)
Gemini-3-Flash	92.00 (+6.50)	92.00 (+4.22)	96.00 (+10.50)	96.00 (+8.22)	62.00 (−23.50)	71.13 (−16.65)
Gemma-4-31B-it	94.00 (+12.00)	94.00 (+1.98)	96.00 (+14.00)	96.00 (+3.98)	44.00 (−38.00)	84.09 (−7.93)
<i>DRC-Reasoning</i>						
GPT-5	63.07 (+31.74)	63.07 (+29.27)	6.00 (−25.33)	7.00 (−26.80)	8.56 (−22.77)	17.74 (−16.06)
Gemini-3-Flash	56.91 (+20.25)	56.91 (+20.16)	18.00 (−18.66)	18.00 (−18.75)	25.71 (−10.95)	26.12 (−10.63)
Gemma-4-31B-it	58.45 (+30.46)	58.45 (+26.40)	2.00 (−25.99)	2.00 (−30.05)	8.56 (−19.43)	25.95 (−6.10)

Δ : Difference from the corresponding overall DRC score in Table 3. †: All rows use the ReAct workflow.

‡: We report three representative backbones here; additional models will be reported in the appendix.

DRC-Essential SR 85.50%, best PPA-Mono SR 64.56%) but collapse on the more practical ones, with PPA-Multi reach only $\sim 31\%$ of PPA-Mono reach and DRC-Reasoning SR only $\sim 43\%$ of DRC-Essential SR. **Geometric reasoning is decoupled from rule recall:** top backbones retain only 34–43% of DRC-Essential SR on DRC-Reasoning, so post-flow violations require coupled geometric planning. **PPA-Mono is target-shortfall:** agents move the right metrics (NIS $>$ SR by 5–12 points) but stop short of the target. **PPA-Multi is trade-off failure:** negative NIS (e.g., Qwen3.5-27B -39.34 ; Gemma-4/ORFS -15.90) reflects constraint regression while chasing the violated objective rather than balancing competing targets. **Scaffold effects are dimension-conditioned.** Proposer–Critic improves some DRC cells by validating edits before dispatch but hurts PPA by over-constraining exploration. ORFS-Agent is complementary: Bayesian optimization lifts Qwen3.5-122B on PPA-Multi (4.00/35.46 $\rightarrow$ 20.00/67.80 SR/NIS) yet amplifies weak inferred search spaces for Gemma-4 (16.34 $\rightarrow -15.90$ NIS). Framework choice is thus not a universal upgrade; it trades off edit validation, search-policy flexibility, and search-space coverage (Section F).

4.2.1 DRC-Bench: per-level breakdown

To localize *which* DRC sub-skill fails, Table 4 decomposes SR and VRR over construction-time levels for three representative ReAct backbones. Parenthesized Δ values report the difference from that model’s overall score in Table 3. **Essential L1/L2 are saturated; L3 is the discriminator.** All three backbones reach 92–96% SR on Essential L1/L2, so atomic rule recall and contextual clutter filtering are largely solved. The L3 cascade drops to 44–62% SR, while VRR remains much higher for GPT-5 and Gemma-4 (54.00/82.78, 44.00/84.09 SR/VRR), showing that agents often fix most violations but leave one residual after interacting edits. This localizes the bottleneck to edit-interaction modeling rather than local rule knowledge. **Reasoning collapses after the first step.** On DRC-Reasoning, L1 remains partially solvable (56.91–63.07% SR), but L2/L3 fall to 2.00–25.71% SR. Gemini-3’s advantage is concentrated exactly in the multi-step regimes (L2/L3: 18.00/25.71 vs GPT-5 6.00/8.56 and Gemma-4 2.00/8.56), not on L1. Thus practical post-flow violations primarily stress trajectory coherence and coupled geometric planning, not single-edit accuracy.

4.2.2 PPA-Bench: per-level breakdown

We focus the main paper on PPA-Performance and PPA-Multi, the two slices that expose re-architecture and multi-objective trade-offs. Table 5 reports SR/NIS for Gemini-3-Flash and Gemma-4-31B-it under ReAct, plus ORFS+Qwen-122B under ORFS-Agent; PPA-Power/Area and the full bar plots are deferred to Section F.4. **PPA-Performance L3 is the re-architecture wall.** L3 drops to only 8–20% SR and 20–27% NIS across all three backbones, with negative deltas for every L3 cell. This isolates RTL micro-architectural reasoning as the bottleneck: agents can improve shallow timing knobs on L1/L2, but fail when retiming or pipelining is structurally required. **PPA-Multi exposes constraint-tracking failure, while ORFS improves coverage.** Moving from L1 (two-objective) to L2 (three-objective) lowers SR/NIS for every backbone; Gemma’s L2 NIS even turns negative (−0.10%), indicating that as the number of competing objectives grows, agents increasingly regress constraint metrics rather than balance the trade-off—the same trade-off bottleneck flagged in the

Table 5: **PPA-Bench per-level performance.** SR/NIS for representative backbones on PPA-Performance and PPA-Multi; parenthesized Δ values report the difference from the corresponding dimension-level overall score. The best score in each column is bolded, with ties sharing bold.

PPA-Performance (Mono)						
	L1		L2		L3	
	SR(%)	NIS(%)	SR(%)	NIS(%)	SR(%)	NIS(%)
Gemini-3-Flash	56.00 (+21.34)	59.99 (+17.70)	40.00 (+5.34)	40.28 (-2.01)	8.00 (-26.66)	26.59 (-15.70)
Gemma-4-31B-it	84.00 (+37.34)	84.00 (+34.58)	36.00 (-10.66)	37.78 (-11.64)	20.00 (-26.66)	26.50 (-22.92)
ORFS+Qwen-122B	20.00 (-6.66)	38.61 (+3.27)	40.00 (+13.34)	47.40 (+12.06)	20.00 (-6.66)	20.00 (-15.34)
PPA-Multi						
	L1		L2		Overall	
	SR(%)	NIS(%)	SR(%)	NIS(%)	SR(%)	NIS(%)
Gemini-3-Flash	25.00 (+6.50)	54.93 (+3.31)	12.00 (-6.50)	48.31 (-3.31)	18.50 (+0.00)	51.62 (+0.00)
Gemma-4-31B-it	17.00 (+6.00)	32.77 (+16.43)	5.00 (-6.00)	-0.10 (-16.44)	11.00 (+0.00)	16.34 (+0.00)
ORFS+Qwen-122B	25.00 (+5.00)	73.85 (+6.05)	15.00 (-5.00)	61.75 (-6.05)	20.00 (+0.00)	67.80 (+0.00)

Δ : Difference from the corresponding dimension-level overall score; negative deltas are red.

†: Gemini-3-Flash and Gemma-4-31B-it use ReAct; ORFS+Qwen-122B uses ORFS-Agent.

‡: PPA-Power/Area figures and analysis are provided in Section F.4.

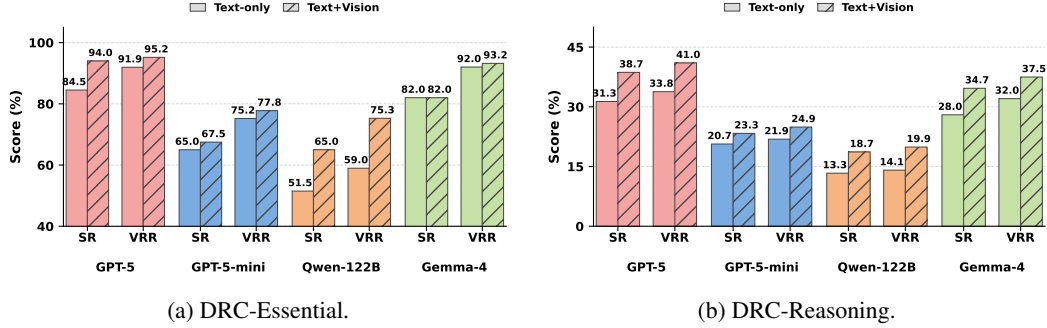


Figure 5: **Effect of vision modality on DRC-Bench.** SR and VRR are combined in each subfigure for four backbones under text-only and text+vision settings.

overall PPA-Multi results. ORFS+Qwen-122B still leads PPA-Multi overall (20.00/67.80 SR/NIS), so structured exploration helps cover the Pareto frontier, but it does not remove the L2 trade-off gap.

4.3 Effect of Vision Modality on DRC-Bench

We pair each text-only backbone (GPT-5, GPT-5-mini, Qwen3.5-122B-A10B, Gemma-4-31B-it) with its vision-augmented variant under ReAct; vision runs add `vision_query_with_pts`, which renders a KLayout image of the violation region (Figure 5). **Vision augmentation enhances DRC-Bench:** across all 16 (model, dimension, metric) cells the layout-image channel never reduces SR or VRR (the only zero-delta is Gemma’s DRC-Essential SR, $82.00 \rightarrow 82.00$) and yields consistent gains on both DRC-Essential and DRC-Reasoning, supplying geometric evidence that text-only rule identifiers and coordinates do not carry. **Compensation is largest when text-only is weak.** Qwen3.5-122B gains +13.5 SR and +16.3 VRR on DRC-Essential from a low baseline (51.50/58.97), while near-saturated GPT-5 and Gemma-4 gain little, so vision compensates for missing geometric evidence rather than amplifying strong text-only agents. **DRC-Reasoning gains are uniform but scale-limited.** SR gains are positive for every backbone (+2.67 to +7.33), with similar VRR gains (+3.04 to +7.22); GPT-5-mini’s smallest lift in every cell (+2.5/ +2.55/ +2.67/ +3.04), despite sharing GPT-5’s visual channel, indicates multimodal fusion capacity is scale-bounded.

4.4 Effect of Iteration Budget

We sweep the iteration cap and compare **Reflexion** [16] on Gemma-4-31B-it under ReAct (Figure 6). **Reflexion protocol.** Reflexion runs two attempts of the per-attempt cap (8 for DRC, 16 for PPA). The second attempt starts *completely fresh* from the original task state—no GDS edits, no flow artifacts,

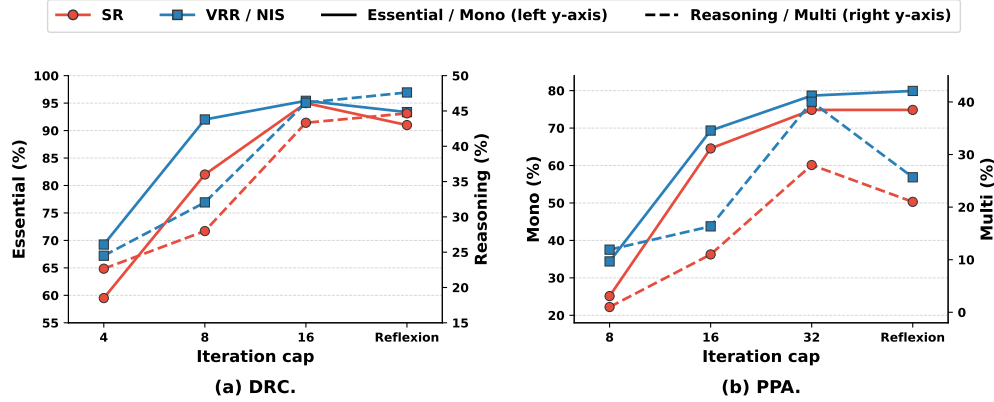


Figure 6: **Effect of iteration cap and Reflexion on Gemma-4-31B-it.** Left: DRC combines DRC-Essential and DRC-Reasoning. Right: PPA combines PPA-Mono and PPA-Multi. Colors encode metrics; solid lines use the left y-axis and dashed lines use the right y-axis.

Table 6: **Thinking-mode ablation** for Gemma-4-31B-it under ReAct.

	Thinking	DRC-Essential		DRC-Reasoning		PPA-Mono		PPA-Multi	
		SR(%)	VRR(%)	SR(%)	VRR(%)	SR(%)	NIS(%)	SR(%)	NIS(%)
React									
Gemma-4-31B-it	On	82.00	92.02	27.99	32.05	64.56	69.34	11.00	16.34
	Off	51.00	63.75	8.66	8.66	54.28	65.20	18.50	38.40
	ToT	85.50	90.35	29.99	31.55	31.42	40.53	0.00	1.35

only a verbal reflection summarized from attempt 1 are carried over (Section E.6). The fair baseline is therefore ReAct at the same per-attempt cap, since each Reflexion trial sees the same per-trial budget; any improvement over that baseline at twice the total cost reflects genuine value from verbal reinforcement, not extra iterations within a single trajectory. **Iteration scaling is dimension-specific.** DRC-Essential and PPA-Mono show diminishing returns, while DRC-Reasoning and PPA-Multi are still climbing at the largest measured caps—one cap saturates easy tasks while leaving hard ones budget-sensitive. **Verbal reflection improves on the per-attempt baseline.** At the per-attempt cap (iter=8/16), Reflexion lifts DRC-Essential SR from 82.00 to 91.00, DRC-Reasoning from 27.99 to 44.66, PPA-Mono from 64.56 to 74.85, and PPA-Multi from 11.00 to 21.00, so the carried-over reflection truly transfers between independent trials.

4.5 Effect of Thinking Mode

We compare standard chain-of-thought (ON), thinking suppressed (OFF), and Tree-of-Thought (ToT) [20] (Section E.6) on Gemma-4-31B-it under ReAct (Table 6). **Thinking is dimension-conditioned.** Turning thinking on improves DRC-Essential SR by 31 points, DRC-Reasoning by 19.3, and PPA-Mono by 10.3, but hurts PPA-Multi (SR/NIS drop from 18.50/38.40 to 11.00/16.34): DRC and PPA-Mono benefit from reasoning before acting, while PPA-Multi benefits more from faster exploration and tool dispatch under a fixed iteration cap. **ToT helps DRC slightly but fails on PPA.** ToT raises DRC SR modestly (+3.5 Essential, +2.0 Reasoning) without improving VRR, suggesting branching closes a few near-complete fixes rather than opening new paths; on PPA it sharply reduces Mono (31.42/40.53 vs. 64.56/69.34 SR/NIS) and nearly fails Multi (0.00/1.35), so extra deliberation is not a substitute for search coverage.

5 Conclusion

We presented POSTEDA-BENCH, a 145-task hierarchical benchmark that jointly evaluates sign-off DRC fixing and PPA convergence. Experiments across eight LLMs under multiple agent scaffolds expose a synthetic-to-practical asymmetry—agents do well on DRC-Essential and PPA-Mono but degrade sharply on DRC-Reasoning and PPA-Multi—and identify trade-off reasoning, rather than knob knowledge, as the PPA-Multi bottleneck; vision augmentation enhances DRC-Bench across the board. POSTEDA-BENCH provides a grounded testbed to drive future research on geometry-aware DRC repair and trade-off-faithful PPA optimization.

Impact Statements

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgments and Disclosure of Funding

References

- [1] X. Chen, M. Lin, N. Schärli, and D. Zhou. Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=KuPixIqPiq>.
- [2] T. Clark, L., V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric. Asap7: A 7-nm finfet predictive process design kit. *Microelectronics Journal*, 53(—): 105–115, 2016. doi: 10.1016/j.mejo.2016.04.006.
- [3] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence. Palm-e: an embodied multimodal language model. In *Proceedings of the 40th International Conference on Machine Learning*, ICML’23. JMLR.org, 2023.
- [4] Y. Fu, Y. Zhang, Z. Yu, S. Li, Z. Ye, C. Li, C. Wan, and Y. C. Lin. Gpt4aigchip: Towards next-generation AI accelerator design automation via large language models. In *IEEE/ACM International Conference on Computer Aided Design, ICCAD 2023, San Francisco, CA, USA, October 28 - Nov. 2, 2023*, pages 1–9. IEEE, 2023. doi: 10.1109/ICCAD57390.2023.10323953. URL <https://doi.org/10.1109/ICCAD57390.2023.10323953>.
- [5] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. Pal: program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, ICML’23. JMLR.org, 2023.
- [6] A. Ghose, A. B. Kahng, S. Kundu, and Z. Wang. Orfs-agent: Tool-using agents for chip design optimization, 2025. URL <https://arxiv.org/abs/2506.08332>.
- [7] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VtmBAGCN7o>.
- [8] Z. Jiang, Q. Zhang, C. Liu, L. Cheng, H. Li, and X. Li. Iicpilot: An intelligent integrated circuit backend design framework using open eda, 2024. URL <https://arxiv.org/abs/2407.12576>.
- [9] M. Liu, N. Pinckney, B. Khailany, and H. Ren. VerilogEval: evaluating large language models for verilog code generation. In *2023 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023.
- [10] M. Liu, T.-D. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu, B. Bhaskaran, B. Catanzaro, A. Chaudhuri, S. Clay, B. Dally, L. Dang, P. Deshpande, S. Dhodhi, S. Halepete, E. Hill, J. Hu, S. Jain, A. Jindal, B. Khailany, G. Kokai, K. Kunal, X. Li, C. Lind, H. Liu, S. Oberman, S. Omar, G. Pasandi, S. Pratty, J. Raiman, A. Sarkar, Z. Shao, H. Sun, P. P. Suthar, V. Tej, W. Turner, K. Xu, and H. Ren. Chipnemo: Domain-adapted llms for chip design, 2024. URL <https://arxiv.org/abs/2311.00176>.
- [11] Y. Lu, S. Liu, Q. Zhang, and Z. Xie. Rtlm: An open-source benchmark for design rtl generation with large language model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 722–727. IEEE, 2024.

- [12] Y. Lu, H. I. Au, J. Zhang, J. Pan, Y. Wang, A. Li, J. Zhang, and Y. Chen. Autoeda: Enabling eda flow automation through microservice-based llm agents, 2025. URL <https://arxiv.org/abs/2508.01012>.
- [13] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [14] OpenCores.org. Opencores: Home, 2026. URL <https://opencores.org/>.
- [15] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: language models can teach themselves to use tools. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*, Red Hook, NY, USA, 2023. Curran Associates Inc.
- [16] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [17] Y. Wang, W. Ye, Y. He, Y. Chen, G. Qu, and A. Li. Mcp4eda: Llm-powered model context protocol rtl-to-gdsii automation with backend aware synthesis optimization, 2025. URL <https://arxiv.org/abs/2507.19570>.
- [18] H. Wu, Z. He, X. Zhang, X. Yao, S. Zheng, H. Zheng, and B. Yu. ChatEDA: A Large Language Model Powered Autonomous Agent for EDA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [19] H. Wu, H. Zheng, Z. He, and B. Yu. Divergent Thoughts toward One Goal: LLM-based Multi-Agent Collaboration System for Electronic Design Automation. In *Annual Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics*, 2025.
- [20] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [21] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [22] K. Zhang, J. Li, G. Li, X. Shi, and Z. Jin. CodeAgent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. In L.-W. Ku, A. Martins, and V. Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13643–13658, Bangkok, Thailand, Aug. 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.737. URL <https://aclanthology.org/2024.acl-long.737/>.

A Additional Related Work

DRC fixing. Before sending a chip layout to the foundry, the physical geometry on each layer must satisfy fabrication constraints provided by the process design kit. These constraints include minimum spacing and width rules, enclosure requirements between layers, and density-related checks. Design rule checking (DRC) verifies whether a layout satisfies these rules, and any violation indicates a potential manufacturability risk that must be resolved before tape-out. Fixing DRC violations is geometry-heavy and often requires local context inspection, root-cause identification, and edits that satisfy the rule without breaking functionality or introducing new violations. Modern EDA toolchains such as Innovus, ICC2, and OpenROAD can resolve many violations automatically, but they may still leave residual sign-off errors or create cascaded violations, requiring manual cleanup by experienced designers.

PPA optimization. Beyond correctness, chip implementation is evaluated by power, performance, and area (PPA). Power captures energy consumption, performance is commonly measured by achievable clock frequency or timing slack, and area reflects silicon cost. These metrics are coupled: improving one objective can degrade another, so practical implementation requires navigating trade-offs. Commercial and open-source EDA tools expose many knobs, including timing constraints, synthesis settings, and place-and-route parameters. Designers specify target PPA goals through configuration files, but automated flows rarely meet all targets in one shot under tight constraints or complex designs. Engineers therefore inspect timing, power, utilization, and area reports, diagnose bottlenecks, and adjust tool settings, constraints, or RTL to converge to acceptable PPA.

B Benchmark Construction Documentation

Filtering and deduplication. The source pool is filtered in the following order: parsing, top-module resolution, synthesis, standard-cell scale, license/release eligibility, and deduplication. Deduplication is performed across all sources rather than within each source separately. We first flag candidates with identical or near-identical top-module names, then compare port signatures (name, direction, and bit width), and finally compare post-synthesis gate-level structure. Designs with more than 90% gate-level overlap are merged, retaining the instance with more complete source files, clearer provenance, or broader downstream task utility.

Contamination control and release package. The source RTL comes from public repositories, so source-code memorization cannot be ruled out. We therefore make the evaluated state depend on benchmark-specific downstream artifacts: violation locations, injected geometries, residual signoff reports, target PPA values, prompts, and tool outputs. The anonymized NeurIPS Evaluations and Datasets submission package contains task definitions, prompts, source RTL, GDS files for DRC-Bench, flow configurations, DRC decks, report parsers, scoring scripts, agent tool interfaces, and machine-readable dataset metadata describing provenance, licenses, intended use, and limitations. For DRC-Bench, released layouts, prompts, sign-off reports, and KLayout setup are sufficient for agent evaluation without regenerating Innovus-derived tasks; for PPA-Bench, each released OpenROAD project directory includes the files, reports, driver, and pinned flow setup needed for re-evaluation.

C DRC-Bench: Reproducibility, Tool Contracts, and Construction Details

C.1 Software stack and tool versions

DRC-Bench is reproduced end-to-end with the following pinned stack:

- **PDK:** ASAP7 [2], the predictive 7 nm process design kit; the bundled DRC deck is used as-is for sign-off.
- **Sign-off DRC:** KLAYOUI v0.30.3 driven by the ASAP7 DRC deck. KLayout alone is sufficient to score any submitted agent.
- **Open-source flow (used to seed DRC-Reasoning L1/L2 and the post-flow layout of DRC-Essential L2):** OPENROAD-FLOW-SCRIPTS; the sweep covers the broad set of tunable variables documented in FlowVariables.md (clock period, utilization, placement effort, routing effort, and many others) rather than a fixed shortlist, so the resulting violation pool is diverse along multiple axes.
- **Commercial flow:** SYNOPSIS DESIGN COMPILER W-2024.09-SP5-3 followed by CADENCE INNOVUS 21.1, with sweeps over the standard synthesis-, placement-, and routing-effort knobs exposed by both tools.
- **Eval driver:** eval/eval_drc_sr.sh. Default configuration: TEMPERATURE=0, NUM_RUNS=5, MAX_PARALLEL_AGENTS=5. The per-task tool-call iteration limit is treated as an evaluation hyperparameter rather than a fixed cap and is varied in our iteration ablation. The driver reports per-task SR (mean over 5 runs) and per-task VRR, aggregated per-class and overall.
- **Agent tools:** released under agents/drc/tools/ (full schemas in Section C.3).

LVS-based functional equivalence is intentionally omitted in this release. The certified ASAP7 LVS rule deck is built for MENTOR CALIBRE v2018, which is no longer broadly accessible to academic

groups; we are migrating DRC-Bench to a more recent open PDK plus a current Calibre release in future work, at which point LVS will be re-introduced as a hard success-criterion gate.

C.2 Per-task layout and release manifest

Every task is a self-contained directory:

- `top_cell.gds`: the input layout supplied to the agent.
- `prompt.txt`: the natural-language prompt embedding the violation list (rule, geometry type, coordinates).
- `info.json`: structured metadata—rule identifier(s), initial violation count, top-cell name, and sign-off DRC runtime.
- `6_drc_count.rpt`: the raw KLayout DRC report used to construct `prompt.txt`.
- `create_errors.py`, `drc_error_collection.py`: the construction scripts that produced the violation, retained for auditability.

Tasks are organized as `benchmark/drc_bench/drc_essential/{L1,L2,L3}/` and `benchmark/drc_bench/drc_reasoning/{L1,L2,L3}/`. The full benchmark, agent tools, and per-task evaluation logs will be released under a permissive license upon acceptance.

C.3 Agent tool contracts

The five tools exposed to the agent have the following contracts (full Python signatures and return schemas are released under `agents/drc/tools/`).

- `query_with_pts(input_gds, level, cell_name, type, direct_layers, context_layers, query_pts, second_level_index_range, third_level_index_range)`: hierarchical textual query at three zoom levels—*first* (shapes directly causing the violation), *second* (shapes inside the bounding box of those shapes), *third* (nearest neighbours just outside the bounding box). `type` is one of `edge-pair`, `polygon`, `edge`; `query_pts` carries coordinates in the layout’s native units. The 38 valid layer names span `Active`, `Gate`, `LIG`, `LISD`, `NSELECT`, `PSELECT`, `SDT`, `M1–M9`, `V0–V9`, and several PDK-specific markers. Returned shapes are sorted by proximity to the violation; pagination over context shapes is exposed via `*_index_range`.
- `vision_query_with_pts(...)`: same query semantics as above but renders a KLayout image of the queried region. The image is converted to RGB JPEG, thumbnail-resized to fit within 1024×1024 at quality 85, Base64-encoded, and returned as a data URI for direct ingestion by vision-language models.
- `add_shape(...)`, `change_shape(...)`, `move_cell(...)`: editing primitives that operate on the working GDS in place; they validate layer membership and shape primitives and reject illegal edits with structured error messages.
- `query_drc_report(...)`: re-runs the KLayout sign-off DRC deck on the current GDS and returns the updated violation list. Each call counts as one iteration against the per-task iteration limit (varied across experiments).

C.4 DRC-Essential rule and layer coverage

DRC-Essential L1 and L2 jointly cover several rule families and 14 layers from the ASAP7 DRC deck, balanced to one representative case per layer per family where the rule admits the layer. Rule families include minimum horizontal/vertical width, minimum horizontal/vertical spacing, minimum enclosed/overlap area, layer-pair extension, and ASAP7-specific quantization rules (e.g., `ACTIVE/SDT` vertical width must be an integer multiple of 27 nm). Two rules from the deck are excluded after pruning topologically equivalent variants; the released `statistic.json` enumerates the per-rule and per-layer task assignments. For L1 construction, examples of topology-specific rules that are hand-drawn by physical-design engineers include `NSELECT.GATE.EX.1–2` and `PSELECT.GATE.EX.1–2`, which require arranging selector and gate shapes to expose the intended extension violation. Examples of rules that admit parametric templates include `WELL.S.2` and

WELL.W.2, where `create_errors.py` instantiates simple rectangles with controlled spacing or width below the ASAP7 threshold.

C.5 Built-in Innovus DRC repair study

To verify that DRC-Reasoning does not simply stop before commercial tools finish, we applied Innovus built-in DRC/ECO repair to five representative post-route designs and then re-ran sign-off DRC. As shown in Table 7, built-in repair substantially reduces but does not eliminate violations; DRC-Reasoning is constructed from this residual class of violations that still require manual or agent-driven geometric repair.

Table 7: **Residual DRC violations after Innovus built-in repair.**

Design	gcd	aes	ibex	jpeg	riscv
Original	34	972	1,551	4,481	9,395
After Innovus repair	22	760	796	1,815	2,750

C.6 DRC-Reasoning step-count labelling protocol

The step-count labels that partition DRC-Reasoning into L1 and L2 are produced by a single physical-design engineer (5+ years of industrial sign-off experience). To control for single-annotator variance we adopt the following written protocol:

1. The engineer loads the violation in KLayout and inspects the offending geometry alongside its surrounding layers.
2. The engineer manually constructs the shortest fix sequence using only the editing operations exposed to the agent (`add_shape`, `change_shape`, `move_cell`); inspection-only tool calls are not counted.
3. The integer count of editing tool calls is recorded as the case’s step count.
4. After a one-week delay the engineer re-labels the same case without access to the prior label; cases with disagreeing labels are re-evaluated under the protocol until convergence, and any case that still admits competing minimal fixes is retained at the smaller step count.

The protocol does not produce inter-annotator agreement scores; it is a within-annotator self-consistency procedure. We disclose this design choice transparently rather than overstate the labelling rigor; broadening to multi-annotator κ scoring is on the future-work list (Section H).

D PPA-Bench: Reproducibility, Tool Contracts, and Construction Details

D.1 Software stack and tool versions

PPA-Bench is reproduced end-to-end with the following pinned stack:

- **PDK:** ASAP7 [2], identical to the DRC-Bench setup.
- **Flow:** OPENROAD-FLOW-SCRIPTS, pinned to commit `a2bb042b6` (release tag `26Q1-127-ga2bb042b6`). The flow is invoked inside a Docker container so that local installations cannot perturb scoring.
- **Eval driver:** `eval/eval_ppa_sr.sh`. Default configuration: `TEMPERATURE=0`, `NUM_RUNS=5`, `TOOL_TIMEOUT=1800s` (30-minute wall-clock cap per `run_openroad` call), `MAX_PARALLEL_AGENTS=5`. The per-task tool-call iteration limit is varied as an evaluation hyperparameter rather than fixed.
- **Scorer:** `benchmark/utis/ppa_score.py`, which implements the SR / NIS computation described in Eq. 2–3.
- **Agent tools:** released under `agents/ppa/tools/` (full schemas in Section D.4).

D.2 Per-task layout

Each PPA-Bench task ships as a self-contained project directory:

- `src/`: the design’s Verilog sources.
- `config/` (a.k.a. `config.mk`, `constraints.sdc`): OpenROAD configuration and SDC constraints, the primary surface the agent edits.
- `logs/`, `reports/`, `objects/`, `results/`: the previous-iteration intermediate artifacts produced by OpenROAD.
- `prompt.txt`: the natural-language prompt summarizing current PPA values, target thresholds, any constraint floors, and a pointer to `FlowVariables.md`.
- `info.json`: ground-truth metadata (initial values, targets, reference knob/value records). Read access by the agent is blocked at the `read_file` layer so that targets cannot be trivially reached by reading the reference fix; the file is retained on disk for auditability and scoring.

Tasks are organized as `benchmark/L3/asap7/single_dimension/openroad/{area, performance, power}/{class1,2,3}` for PPA-Mono and `benchmark/L3/asap7/multi_dimension/openroad/{class1,2}` for PPA-Multi.

D.3 Curated knob list for the parameter sweep

The grid sweep used for PPA-Mono and PPA-Multi construction draws from a curated subset of `eval/FlowVariables.md`. Representative knobs include `ABC_AREA`, `ASAP7_USE_VT`, `CORE_UTILIZATION`, `PLACE_DENSITY`, `SYNTH_HIERARCHICAL`, `TNS_END_PERCENT`, `ROUTE_EFFORT`, and `REPAIR_PDN_VIA_LAYER`. The complete list and the per-knob value ranges used in our sweep are released alongside the benchmark.

D.4 Agent tool contracts

The four PPA-Bench tools have the following contracts (full Python signatures and return schemas are released under `agents/ppa/tools/`).

- `read_file(command)`: executes a shell-style read command inside a per-task sandbox. A whitelist of safe commands is permitted (`cat`, `head`, `tail`, `grep`, `awk`, `sed`, `wc`, `sort`, `uniq`, `cut`, `tr`, `ls`, `find`, `tree`, `jq`, `less`, `more`, `file`, `stat`, `diff`, `strings`, `xxd`, `od`); modifying commands and shell control operators (`>`, `>>`, `&&`, command substitution, etc.) are blocked. Pipes are allowed. Reads of `info.json` or paths outside the sandbox are rejected.
- `edit_file(path, old_content, new_content)`: in-place replacement edit on files inside the sandbox. A small set of structural variables in `.mk` files (`PLATFORM`, `DESIGN_NAME`, `DESIGN_NICKNAME`, `VERILOG_FILES`, `SDC_FILE`) is locked to prevent flow-breaking modifications; all other configuration entries, SDC constraints, and Verilog sources are editable.
- `run_openroad(...)`: launches an OpenROAD flow run on the current sandbox, returning `stdout/stderr` and exit status. Each call is bounded by a `TOOL_TIMEOUT` environment variable. A flow crash returns the error to the agent and counts as one iteration.
- `report_ppa()`: parses the latest reports and returns a structured summary of effective period, total power, and die area together with the gaps to the per-task targets.

D.5 Functional-equivalence sanity check

PPA-Bench evaluates RTL-level optimization, so an agent that meets every PPA target by silently breaking the design is not credited. A task is therefore counted as successful only when, in addition to satisfying all PPA targets, its modified RTL passes a functional-equivalence sanity check against the original sources. The check is implemented in `eval/equiv/check_equiv.sh` and is intentionally framed as a *sanity gate* rather than formal verification, because the most common L3 interventions in PPA-Mono (pipelining, retiming, datapath restructuring) legitimately change cycle latency and handshake timing and would be flagged by a strict combinational equivalence checker.

For each design we ship a hand-written testbench under `eval/equiv/tb/<design>_tb.v` that drives a representative input sequence and emits cycle-stamped output traces. The original and modified sources are both compiled with ICARUS VERILOG (IVERILOG_LANG=2012) and simulated against the same testbench. We then compare traces in one of two modes, selected per design:

- **Event-matched mode.** For combinational and untimed designs, traces are diffed line-by-line; any divergence fails the check.
- **Latency-sweep mode.** For pipelined or handshake-timed designs, a separate comparator (`eval/equiv/compare.py`) sweeps a latency offset $L \in [0, L_{\max}]$ (default $L_{\max} = 16$ cycles, with an 8-cycle reset warmup) and checks whether $\text{orig}[t] = \text{mod}[t - L]$ over the overlap window for some L . The check passes if any offset yields zero mismatches, so legitimate pipeline-depth or handshake-timing changes do not count as functional regressions.

The check returns four exit codes: 0 = equivalent (or skipped because no testbench exists for this design), 1 = output mismatch (the agent broke functionality), 2 = build failure on either the original or modified design, 3 = usage/internal error. Designs without testbenches—typically processor cores such as `ibex_core` that exercise SystemVerilog features Icarus Verilog does not support—are skipped, and we explicitly disclose this in our task-level results so reviewers can audit which tasks are functionally gated.

E Agent Frameworks and Experimental Protocol

E.1 Models and inference setup

The eight models reported in Table 3 are evaluated under a shared agent codepath so that any cross-model comparison reflects model and scaffold capability rather than client-side variation.

- **Commercial.** GPT-5, GPT-5-mini, Gemini-3-Flash-preview, accessed via their official APIs at the time of evaluation. The exact API model identifiers are recorded in the per-run cost-tracker logs released with the benchmark.
- **Open-source.** DeepSeek-V3.2, Qwen3.5-122B-A10B, Qwen3.5-27B, Gemma-4-31B-it, and Qwen3.5-9B, each served via vLLM on a local GPU cluster behind an OpenAI-compatible /v1 endpoint. Tensor-parallel degree is selected per checkpoint to fit GPU memory; greedy decoding (`temperature=0`) is enforced server-side.

Token usage and tool-call counts are tracked per run via a shared cost-tracking utility shipped alongside the agent code.

E.2 Iteration semantics per framework

We adopt a per-framework definition of one iteration so that the iteration cap (8 for DRC-Bench, 16 for PPA-Bench) is interpreted consistently:

- **ReAct.** One iteration is one tool call dispatched by the agent. Pure reasoning turns that do not emit a tool call are not counted.
- **Proposer–Critic.** The proposer drafts a tool call as a textual description; the critic validates or refines the proposal; only the agreed action is dispatched. The full proposer–critic–dispatch triple counts as one iteration; proposal rounds that do not lead to a dispatch are not counted.
- **ORFS-Agent.** ORFS-Agent uses a fixed Bayesian-optimization budget of 6 outer iterations $\times$ 3 parallel candidates per outer iteration = 18 OpenROAD candidate runs per task, hardcoded in the wrapper rather than driven by the global iteration cap. Each candidate run is one OpenROAD flow execution; the single LLM call consumed during the search-space-discovery setup phase is not counted against the budget.

Adaptive iteration cap. The headline caps in Section 4.1 (8 DRC, 16 PPA) are stated in units of *agent actions* (tool dispatches), not raw LLM calls. Multi-agent scaffolds that consume additional internal LLM calls per dispatch—e.g., Proposer–Critic’s proposer→critic→dispatch triple, or ToT’s

per-node candidate sampling and judge scoring—therefore receive a proportionally larger total LLM-call budget than ReAct’s raw tool-call budget. We scale the underlying LLM-call budget linearly with the number of cooperating agents per dispatch, so that adding more agents to a workflow grants correspondingly more reasoning calls. This adaptive scaling keeps cross-scaffold comparisons about *scaffold value* rather than penalising frameworks for needing more thought per action.

E.3 Run termination and timeout handling

A run terminates when (i) the success criterion is met, (ii) the iteration cap is reached, (iii) the underlying tool times out, or (iv) the LangGraph recursion limit is hit; in all non-success cases the partial-progress metric is computed from the last valid state. The default `TOOL_TIMEOUT` is 1800 s for OpenROAD flow runs and the equivalent for KLayout DRC. Tool timeouts and flow crashes return the error to the agent and count as one iteration.

Excluded baselines. Most prior EDA-LLM benchmarks (Table 1) target script generation from natural language rather than PPA convergence or DRC fixing, and their agents do not expose a comparable success criterion; we therefore exclude them from Table 3. ORFS-Agent [6] is the closest published baseline for PPA optimization and is included via the wrapper described below.

E.4 ORFS-Agent search-space-discovery wrapper

The published ORFS-Agent [6] requires every (design, PDK) pair to ship a hand-curated `opt_config.json` that lists the tunable parameters and their ranges, plus per-design baseline ECP and wirelength values. Our PPA-Bench tasks are driven by natural-language prompts and do not carry this metadata, so we adapt the published agent with an LLM-based setup phase that automates the configuration step. Concretely:

- A single LLM call reads the task’s `prompt.txt`, the contents of `config/` (i.e., `config.mk` and `constraints.sdc`), the list of files under `src/`, and `eval/FlowVariables.md`, and returns a structured search space (tunable parameters, ranges, defaults) via a single tool call. Targets and the optimization objective are not picked by the LLM—they are parsed from `info.json` by the orchestrator outside the agent’s view—so the discovery step affects only *which knobs to sweep*, not the success criterion.
- The discovered search space is fed into the original GP-driven outer loop: iteration 1 uses Latin Hypercube initialization around the discovered defaults; iterations 2– N propose new candidates by maximizing a UCB acquisition over a Matérn-kernel GP, with diversity-aware top- k selection. Each candidate runs a real OpenROAD flow inside an isolated per-candidate sandbox.
- Compared to upstream ORFS-Agent (6 outer iterations $\times$ 50 parallel candidates $\approx$ 300 runs/task), our adaptation uses a smaller budget (6 outer $\times$ 3 parallel = 18 runs/task) so that wall-clock cost is comparable to ReAct and Proposer–Critic on the same hardware.
- The eval driver counts each candidate ORFS run as one iteration against the per-task budget (16 for PPA-Bench).

The wrapper preserves the upstream agent’s CLI contract so that `eval_ppa_sr.sh` can swap it in for ReAct or Proposer–Critic without modification.

E.5 Framework-level system prompts

We release the system prompts used by ReAct and Proposer–Critic for both DRC-Bench and PPA-Bench under `agents/{drc,ppa}/{react,proposer_critic}/`. The prompts are held fixed across models within a framework so that benchmark scores reflect model and scaffold capability rather than prompt engineering. Each prompt declares (a) the role and tool repertoire available to the model, (b) the success criterion (DRC-clean for DRC-Bench, target metrics + functional equivalence for PPA-Bench), and (c) framework-specific scaffolding—chain-of-thought interleaving for ReAct, role separation between proposer and critic for Proposer–Critic, and the structured search-space schema for the ORFS-Agent setup phase.

E.6 Reflexion and Tree-of-Thought wrappers (PPA)

We also release PPA-side Reflexion and Tree-of-Thought wrappers used in the iteration-budget and thinking-mode ablations (Sections 4.4 and 4.5); both are layered on top of the same ReAct PPA primitives and tool surface so that scores reflect the search/reflection policy rather than tool plumbing.

Reflexion (`agents/ppa/reflexion/`). The wrapper implements the verbal-reinforcement trial-and-reflect loop of Shinn et al. [16]. The user-visible iteration budget is divided evenly across N trials (we use $N=2$, matching the halve-and-reflect-once protocol of Section 4.4). Each trial is a full ReAct rollout with a freshly initialised conversational memory; between trials the agent makes a tool-less Self-Reflection LLM call that summarises lessons from the previous trajectory, and these lessons are appended to the system prompt of the next trial as long-term memory. The Self-Reflection call does not consume iteration budget. To match canonical Reflexion semantics, the working sandbox is snapshotted before trial 1 and restored before every subsequent trial, so each trial sees the original task content; only the verbal reflection is carried across. Cumulative tool-call counts and token costs are aggregated across trials so the eval driver (`eval_ppa_sr.sh`) sees a single agent-run summary, matching the ReAct-agent contract.

Tree-of-Thought (`agents/ppa/tot/`). The wrapper implements the ToT search of Yao et al. [20] over the same tool surface as the ReAct PPA agent. At each tree node the agent samples `PARALLEL_NODE` candidate next actions in parallel, an LLM judge scores them, the highest-scored candidate is executed, and the resulting state becomes a new node. Un-executed candidates from every previously-expanded node remain in a global priority queue, so when the current path stalls the search backtracks to the highest-scored alternative from an ancestor. Iteration budget is the total tool-call budget across the whole tree; `max_depth` caps the deepest single path (default 16, matching ReAct’s per-run cap), and per-step LLM calls (candidate sampling and scoring) are billed in tokens but do not consume iteration budget. Each node owns a sandbox snapshot under `$AGENT_RESULT_DIR/_tot_snapshots/`; on backtrack, the sandbox is wiped and restored from the chosen ancestor’s snapshot before executing. At end of run the sandbox is restored to the best node, ranked by (i) success, (ii) at least one successful `run_openroad_flow` call, (iii) deeper paths over shallower ones, and (iv) judge score, so the eval driver picks up a node whose OpenROAD reports actually exist.

F Extended Analysis of Main Results

This section expands on the deductions in Section 4.2, focusing on framework-level effects and cross-dimension comparisons that are too detailed to fit inline.

F.1 Scaffold value is dimension-conditioned and reverses sign on PPA

Switching from ReAct to Proposer–Critic raises Qwen3.5-122B’s DRC-Essential SR by +15 points and Gemma-4-31B-it’s DRC-Reasoning SR by +3.3, but cuts Gemma-4’s PPA-Mono SR by 22 points ($64.56 \rightarrow 42.28$) and Qwen3.5-9B’s by 21.7. Since our evaluation uses adaptive, approximately matched budgets across frameworks, this reversal is not explained by fewer allowed optimization steps. Instead, it suggests that proposer–critic validation changes the search policy: on DRC tasks where each call is a local geometric edit, the validator catches malformed edits before dispatch and pays for itself; on PPA tasks, where progress depends on empirical `run_openroad` measurements, critic-side filtering can bias the trajectory toward conservative local changes and reduce broad parameter exploration. Framework choice is therefore not a global hyperparameter, and cross-framework comparisons that aggregate over dimensions will mislead.

F.2 Bayesian optimization amplifies the LLM’s search-space prior

ORFS-Agent lifts Qwen3.5-122B’s PPA-Multi NIS from 35.46 to 67.80 (the table’s highest value, $\sim 1.9\times$) but drives Gemma-4-31B-it’s PPA-Multi NIS from +16.34 to −15.90 and its PPA-Mono NIS from 69.34 to 47.31. Both runs share the same outer BO loop and budget; the only difference is the LLM-induced search space inferred from `FlowVariables.md`. The Pareto-frontier failure of pure ReAct is therefore best understood as a poor exploration policy layered on largely correct knob

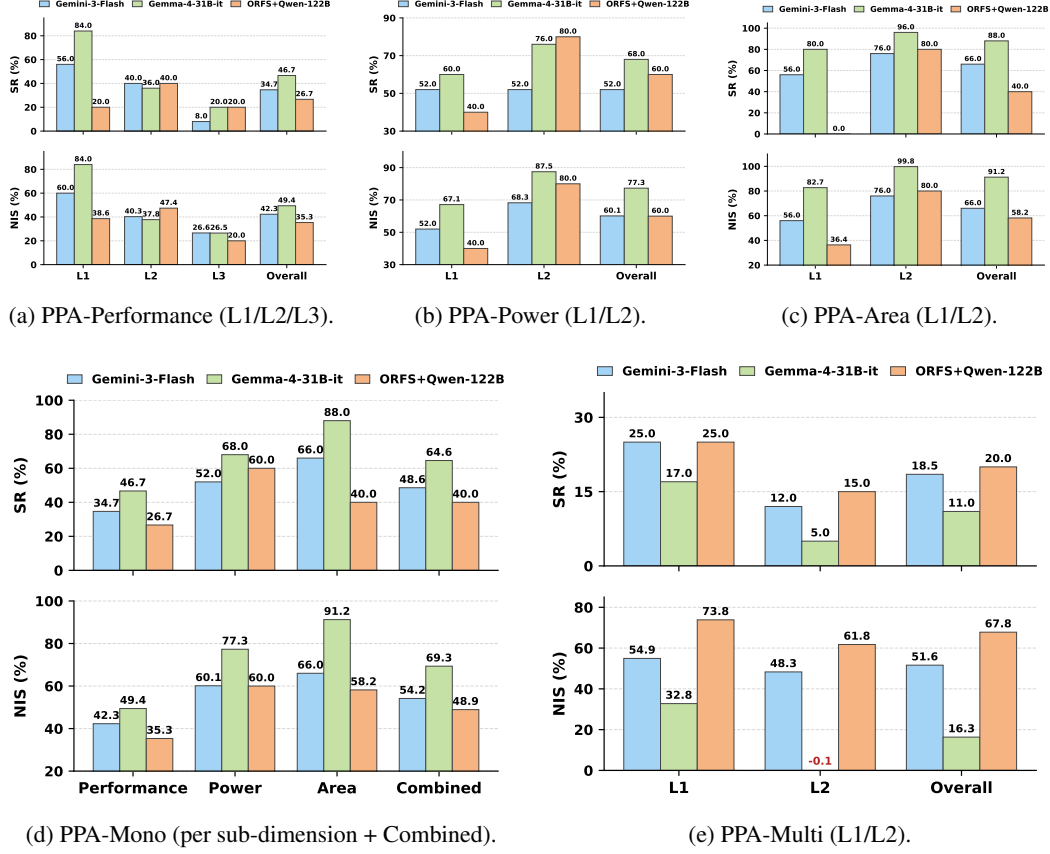


Figure 7: **Full PPA-Bench per-level performance.** SR (top) and NIS (bottom) per construction-time level. Row 1 decomposes PPA-Mono into Performance / Power / Area; row 2 summarizes PPA-Mono per sub-dimension and reports PPA-Multi.

knowledge: when a structured sampler is grafted on top of a competent space, multi-objective SR jumps to first place; when it is grafted on top of a wrong space, it amplifies the error rather than correcting it. The open research problem is LLM-side search-space discovery, not the optimization loop itself.

F.3 Capability-gap ordering on POSTEDA-BENCH

Top-cell ratios across dimensions give a quantitative difficulty ordering: PPA-Multi reach is $20.00/64.56 = 31\%$ of PPA-Mono reach, while DRC-Reasoning reach is $36.66/85.50 = 43\%$ of DRC-Essential reach. Multi-objective trade-off is therefore the largest absolute capability gap today, ahead of geometric DRC reasoning. This suggests that the next-step research direction is not better DRC-rule prompting but tool-augmented planners with explicit constraint-tracking memory.

F.4 Additional PPA-Bench per-level breakdowns

Figure 7 gives the full five-panel PPA decomposition used to support the compact main-text table in Table 5.

Power/Area show inverted L1→L2. Both PPA-Power and PPA-Area have $L2 \geq L1$ SR for every model (e.g., ORFS+Qwen $40 \rightarrow 80\%$ on Power and $0 \rightarrow 80\%$ on Area; Gemma $60 \rightarrow 76\%$ on Power). This counter-intuitive ordering suggests that joint power/area perturbations create slack: additional misconfigured knobs offer more recovery levers, whereas single-knob cases require identifying the responsible knob exactly.

The full decomposition clarifies ORFS behavior. ORFS+Qwen leads PPA-Multi NIS by a wide margin (67.80 vs Gemini 51.62 and Gemma 16.34) and matches or exceeds the others on Power L2 / Area L2, but trails the PPA-Mono Combined score (40.00/48.91 vs Gemma 64.56/69.34) and has 0% SR on Area L1. This supports the main-text conclusion that Bayesian optimization helps through exploration coverage when the inferred search space contains useful knobs, but can amplify a weak inferred space when the task hinges on a specific single knob.

G Detailed Ablation Readings

We expand on the ablation figures in Sections 4.3 to 4.5. Source tables and per-run logs accompany the released benchmark.

Detailed iteration-budget reading. Figure 6 separates two regimes. On DRC-Essential and PPA-Mono, SR follows a concave shape ($59.5 \rightarrow 82.0 \rightarrow 95.0$ and $25.1 \rightarrow 64.6 \rightarrow 74.9$), so extra iterations mostly polish already reachable tasks. DRC-Reasoning and PPA-Multi behave oppositely: Reasoning gains only +5.3 SR from $4 \rightarrow 8$ but +15.3 from $8 \rightarrow 16$, while Multi gains +10 from $8 \rightarrow 16$ and +17 from $16 \rightarrow 32$. The PPA-Multi NIS—SR gap also narrows then widens ($10.91 \rightarrow 5.34 \rightarrow 12.04$), indicating that additional iterations can improve the violated metric without crossing the success boundary. DRC-Essential shows the opposite pattern: its VRR—SR gap collapses from $9.75 \rightarrow 10.0 \rightarrow 0.42$, consistent with all-or-nothing cascaded fixes once enough edit budget is available.

H Limitations

We highlight the limitations of the current release so that users can interpret scores correctly.

- **DRC-Bench: single PDK.** All DRC tasks target ASAP7. Generalization to commercial advanced-node PDKs (e.g., TSMC/Samsung) is untested and bound by NDA, although the construction pipeline is PDK-agnostic.
- **DRC-Bench: single annotator for step-count labels.** DRC-Reasoning L1/L2 step counts come from one engineer with a self-consistency protocol. Inter-annotator agreement scoring with a second physical-design expert is planned.
- **PPA-Bench: OpenROAD-only.** The current PPA suite is built exclusively on OPENROAD-FLOW-SCRIPTS. We plan to extend the suite with SYNOPSIS DC + CADENCE INNOVUS tasks in a future release, mirroring the dual-flow coverage already established for DRC-Bench.
- **PPA-Bench: wall-clock cost.** Each `run_openroad` call is a full flow execution and consumes minutes-to-tens-of-minutes of wall-clock per call; large-scale agent comparisons are correspondingly expensive.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction state POSTEDA-BENCH’s scope (DRC-Bench and PPA-Bench, four sub-dimensions, 145 tasks across academic and commercial toolchains) and key findings; these are directly supported by the construction details in Sections 3.1, 3.1.2 and 3.1.3 and the experimental results in Sections 4.2, 4.2.1 and 4.2.2.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: A dedicated Limitations section (Section H) discusses the absence of an LVS gate, single-PDK coverage (ASAP7), single-annotator step-count labels, OpenROAD-only PPA tasks, dataset-internal target selection, and the wall-clock cost of full-flow runs.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper introduces benchmark metric definitions (Violation Reduction Rate and Normalized Improvement Score) but does not claim any theoretical result that requires a proof.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections C to E document pinned tool versions (ASAP7, KLayout v0.30.3, OpenROAD-flow-scripts, DC W-2024.09-SP5-3, Innovus 21.1), per-task release manifests, eval drivers (`eval_drc_sr.sh`, `eval_ppa_sr.sh`), agent tool contracts, iteration semantics, and termination/timeout rules; sign-off DRC labels are machine-checkable via KLayout and PPA labels via OpenROAD report parsers.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: An anonymized Evaluations & Datasets package accompanies this submission containing task definitions, prompts, source RTL, GDS files for DRC-Bench, flow configurations, DRC decks, report parsers, scoring scripts, agent tool interfaces, and machine-readable dataset metadata (Sections B and C.2); the full benchmark, agent tools, and per-task evaluation logs will be released under a permissive license upon acceptance.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 4.1 states the eight evaluated models, three agent scaffolds (ReAct, Proposer-Critic, ORFS-Agent), 5-run temperature-0 protocol, and iteration caps (8 DRC / 16 PPA, 18 OpenROAD candidates for ORFS); Sections E.1 to E.5 provide vLLM serving setup, per-framework iteration semantics, timeout handling, the ORFS search-space-discovery wrapper, and released system prompts.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: Reported SR/VRR/NIS values are means over 5 independent temperature-0 runs per (model, framework, task); explicit error bars are omitted because each run is a costly tool-in-the-loop trajectory whose dominant cost is full EDA-flow execution—a single OpenROAD flow takes minutes to tens of minutes (Section H). Per-run scores will be released alongside the benchmark to support post-hoc variance analysis.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.).
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Open-source backbones are vLLM-served on a local GPU cluster of 8×NVIDIA H200 and 8×NVIDIA RTX 6000 Pro with tensor-parallel degree selected per checkpoint to fit GPU memory; commercial models (GPT-5, GPT-5-mini, Gemini-3-Flash-preview) are accessed via their official APIs (Section E.1). EDA-side cost is dominated by full-flow runs: each OpenROAD invocation has a 1800 s timeout and ORFS-Agent uses 18 candidate flow runs per PPA task (Sections E.3 and E.4).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: The work uses public RTL sources (RTL2M v2, VerilogEval-Human, OpenCores) under their published licenses with citations (Section B), evaluates LLMs via their official APIs or self-hosted weights, involves no human subjects, and preserves submission anonymity per the NeurIPS guidelines.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Impact Statement at the end of the main text and the Limitations section (Section H) discuss the benchmark's role in advancing reliable EDA-agent research and the constraints on generalization (single PDK, OpenROAD-only PPA, etc.); we identify no direct path to malicious use beyond standard EDA tooling.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released artifacts are GDS layouts, OpenROAD project directories, sign-off DRC reports, prompts, and evaluation drivers built on a predictive academic PDK (ASAP7); they carry no high-risk content (e.g., pretrained generative models, scraped imagery, or personal data).

Guidelines:

- The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: All upstream assets are cited in the main text and appendix: RTL sources (RTL2LM v2 [11], VerilogEval-Human [9], OpenCores [14]), the ASAP7 PDK [2], and the EDA stack (KLayout v0.30.3, OpenROAD-flow-scripts, Synopsys DC W-2024.09-SP5-3, Cadence Innovus 21.1) are documented with versions and provenance in Sections B and C.1; the per-design release manifest records license and provenance.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: POSTEDA-BENCH is released with per-task manifests (Sections C.2 and D) that bundle the input GDS or OpenROAD project, prompt, structured `info.json`, raw DRC reports, construction scripts, and a machine-readable dataset card describing provenance, intended use, and limitations; tool contracts and system prompts are documented in Sections C.3, D.4 and E.5.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The work uses no crowdsourcing platforms and conducts no human-subjects research; the DRC-Reasoning step-count labels are produced by a single in-house physical-design engineer following the written self-consistency protocol in Section C.6.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The work involves no human-subjects research, so IRB approval does not apply.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are the subjects of evaluation rather than a core methodological component of POSTEDA-BENCH; the evaluated commercial and open-source models, agent scaffolds, and inference setup are documented in Sections 4.1, E, E.1 and E.5.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.